

Auto-Echo: Automated Discovery of Cache-Hierarchy Structure from User-Space Latency Alone

Harsh Raj Singh

MSc Advanced Computer Science
Queen Mary University of London
London, United Kingdom
ec25303@qmul.ac.uk

Abstract—Can a program discover its machine’s cache hierarchy — how many levels it has and how large each is — from timing alone, with no privileges, no vendor tables and no architecture-specific instructions? The usual answer is to read a table somebody else has filled in, but that table can be incomplete, and it can be complete and still not describe how the machine behaves. This paper presents Auto-Echo, which answers the question by measurement. A flush-free pointer-chase probe with batch-amortised, runtime-calibrated timing produces a latency-versus-working-set-size curve; an unsupervised stage then determines the hierarchy from it, with exact one-dimensional clustering counting the memory levels and count-constrained change-point detection localising each capacity. No threshold, penalty or level count is supplied anywhere in the productive path, and the counting step is solved to global optimality by dynamic programming rather than by a local-search heuristic, making it deterministic. On an Apple M1 and an Intel Core i5-13450HX the same code path recovers every documented cache within a factor of two — 2/2 and 3/3 respectively, at 100% precision — against operating-system ground truth that is itself cross-checked against an independent implementation. Two boundary conditions are established by controlled experiment rather than asserted. Page size, not cache size, decides whether a last-level cache is visible at all: the Intel machine’s documented 20 MiB L3 is behaviourally unreachable under default 4 KiB pages and resolves at 19.7 MiB only under a 2 MiB huge-page allocation. And what a single core recovers of a *shared* cache is not its capacity but the share left to it, which an eight-core streaming load drives down by a factor of five. The M1 result includes a cache no operating-system interface reports.

Index Terms—cache hierarchy, memory latency, performance measurement, unsupervised learning, change-point detection, computer architecture

I. INTRODUCTION

Every program’s performance depends on the shape of the cache hierarchy it runs on: how many levels exist, and how large each one is. Wulf and McKee’s “memory wall” (Wulf and McKee 1995) named the structural cause — processor and memory speeds improving at different exponential rates — and Drepper’s practitioner survey (Drepper 2007) draws the corollary that performance on modern hardware is largely decided by how a program’s working set maps onto that hierarchy. Knowing the hierarchy’s shape is therefore a precondition for

reasoning about performance, yet the shape itself is largely opaque to unprivileged software.

The standard response is to consult a table that somebody else has already filled in. Three sources dominate, and it is worth being precise about what each actually is, because the case for measurement is easy to overstate. The first is the *commercial* view: the vendor’s own system report. The second is the *software* view: the operating system’s cache descriptors, read through a library such as `hwloc` (Broquedis et al 2010), an architecture-specific instruction such as the x86 `CPUID` deterministic cache-parameter leaf (Intel 2023), or a vendor tool such as Intel’s Memory Latency Checker (Intel 2024). The third is measurement.

This paper reports what the same two machines — an Apple M1 and an Intel Core i5-13450HX — look like through all three lenses, and the three do not agree.

The commercial lens is the one most users ever consult, and on both machines it reports the processor, its core count and its installed memory, and *not one figure about the cache hierarchy*. Anyone who needs the size of L1 has already been forced to a second tool.

The software lens is that second tool, and on hardware whose vendor filled in the table it is excellent. On the Intel machine `lstopo` reports a complete and correct hierarchy — 48 KiB L1d and 1280 KiB L2 per performance core, a 20 MB shared L3 — which independent measurement then confirms. Nothing here disputes that, and an argument for measurement resting on the descriptor being *inaccurate* would simply be false. Its limitation is narrower and more interesting: its chain of trust terminates at what the operating system was told, so it inherits the firmware’s omissions exactly.

That limitation bites in two distinct ways, and this work exists because of both.

The table can be incomplete. On the Apple M1, `lstopo` reports the 128 KB L1d and the 12 MB L2 and then stops. The System-Level Cache is absent — not misreported, *absent* — because Apple exports it through no interface `hwloc` can read. Section IV shows the probe responding to capacity well beyond the documented 12 MB L2, resolving the L2 and the SLC as a single 13.9 MiB band. What was never written down

cannot be looked up; it can only be measured.

The table can be complete, yet topology is not behaviour. On the Intel machine `lstopo`'s 20MB L3 is entirely correct as a statement about the silicon — and under the default 4 KiB page size that L3 is behaviourally *unreachable*. Past roughly 1.3 MiB the measured curve climbs and saturates at a flat ~ 143 ns plateau by 4–5 MiB, because once the working set outruns the translation lookaside buffer's reach every dependent load triggers a page-table walk that itself misses to DRAM. The cache is present; the machine cannot reach it before translation cost dominates. A 2 MiB huge-page allocation lifts the masking and the same code path then recovers the L3 at 19.7 MiB, within -1.5% of nominal.

The second case is the sharper of the two, because no descriptor is wrong in it. A table reports what a cache *is*; it cannot report what a workload can *use*, and the gap between those is a property of the running system — page size, TLB reach, and, as Section IV shows under load, how much of a shared cache the other cores have left. Measurement is therefore not a less accurate substitute for the descriptor. It answers a different question.

Contributions. This paper makes four.

- 1) A portable, flush-free working-set-size pointer-chase probe with batch-amortised timing and a tick-to-nanosecond conversion *calibrated at runtime*, removing any dependence on a configured CPU frequency (Section III).
- 2) An automatic, penalty-free level-discovery stage in which clustering *counts* the memory levels and change-point detection *localises* each capacity, with the counting step solved to global optimality by dynamic programming rather than by Lloyd's heuristic (Section III).
- 3) A self-validating evaluation on two real ISAs, in which every documented cache on both machines is recovered and matched against operating-system ground truth that is itself cross-checked against an independent implementation (Section IV).
- 4) Two measured boundary conditions on what user-space discovery can achieve: page size, not cache size, decides whether a last-level cache is visible at all, and what a single core recovers of a *shared* cache is not its capacity but the share left to that core (Section IV).

II. RELATED WORK

A. Classical User-Space Cache Characterisation

Inferring cache parameters from timing is long-established. Saavedra and Smith (1995) characterised machines by timing loops over strided arrays, extracting cache size, line size and associativity from the resulting surface. McVoy and Staelin's *Imbench* (McVoy and Staelin 1996) made the technique a standard tool: its `lat_mem_rd` benchmark walks a pointer chain over a range of working-set sizes and reports a latency curve whose plateaus correspond to cache levels. Yotov et al (2005) automated parameter extraction from such curves for compiler use, and the memory-mountain presentation familiar

from Bryant and O'Hallaron (Bryant and O'Hallaron 2015) is the same measurement viewed as a surface over size and stride.

This lineage establishes the measurement primitive that the present work also uses, and no novelty is claimed for it. What that literature does not provide is an *inference* layer: the number of levels is supplied by the experimenter, who knows what machine is being measured, and the boundaries are read off the curve by eye or by a threshold tuned to the machine at hand. The contribution here sits at that layer rather than beneath it.

B. Microarchitectural Side Channels

A second literature recovers cache geometry from unprivileged timing far more precisely than anything attempted here, and it would be misleading to present timing-based inference as an open problem while ignoring it. Percival (2005) showed that cache timing alone leaks a co-resident process's access pattern. Osvik et al (2006) formalised *Prime+Probe*, in which an attacker fills a cache set and times its own re-access; Yarom and Falkner's *Flush+Reload* (Yarom and Falkner 2014) inverted the mechanism using `clflush` to achieve single-line resolution on the last-level cache. Eviction-set construction (Vila et al 2019) recovers set structure and associativity with an exactness a latency sweep cannot match.

These methods are more precise than this work on the caches they target, and if the goal were associativity or line size they would be the better instrument. Two things put them outside the present setting. They generally require knowledge of the target — which sets to prime, which lines to flush — and they depend on primitives that are not portable: `clflush` is x86-only, and macOS on Apple Silicon exposes no user-space data-cache flush at all. A method that must run on an undocumented machine with no privileges cannot assume either.

One result from this literature bears directly on the method's core assumption. Vicarte et al (2022) demonstrated a *data memory-dependent* prefetcher on the Apple M1 that follows pointer-like loaded values, and Chen et al (2024) built an attack on it. A pointer chase is precisely the access pattern such a unit targets, which is a genuine threat to a randomised pointer-chase probe on Apple Silicon specifically. Section IV returns to this with the measured evidence.

C. Topology Tools and the Chain of Trust

The tools practitioners actually reach for do not measure at all. `hwloc` (Broquedis et al 2010) assembles a topology tree from what the operating system and firmware expose. The x86 `CPUID` deterministic cache-parameter leaf (Intel 2023) returns level, type, line size and associativity directly, and is architecture-specific by construction. Intel's Memory Latency Checker (Intel 2024) is a vendor instrument for vendor hardware. Privileged interfaces — performance counters through PAPI (Terpstra et al 2010) or LIKWID (Treibig et al 2010), or a kernel module — extend the reach further still.

Each is more accurate than measurement on the hardware it covers, and each fails in the same way: it presupposes that the hierarchy has already been described somewhere. The distinction that matters for this paper is that `hwloc` is an independent *implementation*, not an independent *source*. It reads the same operating-system interfaces any other reader would, so where the OS is silent it is silent too. Section IV uses it in exactly that capacity: as a check that the ground truth is being *read* correctly, which is a different claim from the ground truth being *complete*.

D. Unsupervised Model Selection

Deciding how many levels a curve contains is a model-selection problem. The clustering objective and the segmentation objective are both monotonically non-increasing in the number of components, so neither can select its own — minimising either returns the degenerate solution in which every observation is its own cluster. Three families of remedy exist: penalise complexity, as PELT does for change-point detection (Killick et al 2012); look for a knee in the cost curve, as the elbow method does; or score a criterion that is not monotone, such as the silhouette coefficient (Rousseeuw 1987). Only the third has an interior optimum and therefore needs no threshold, and it is the one adopted here.

For the counting step itself, one-dimensional k -means admits an exact solution. An optimal partition of points on a line is necessarily contiguous in sorted order, which reduces the search from a Stirling number of partitions to a choice of cut points and permits an exact dynamic program — Fisher’s grouping for maximum homogeneity (Fisher 1958), known to cartography as Jenks natural breaks and implemented as `Ckmeans.1d.dp` by Wang and Song (2011). Grönlund et al (2017) later reduced the bound to $O(n \log n)$. Using an exact solver in place of Lloyd’s heuristic (Lloyd 1982) makes the level count provably optimal and fully deterministic, which matters because the evaluation reports variability across sweeps and any solver-induced variance would contaminate a hardware measurement.

E. Positioning

Measuring cache capacities by pointer chasing is classical, and claiming it as novel would be indefensible. The contribution of this work is the layer above the measurement: a fully unsupervised, zero-configuration inference stage that determines how many levels exist and where their boundaries lie with no hard-coded thresholds and no prior knowledge of the machine, and that validates itself automatically against operating-system ground truth on whatever machine it runs on. Where Yotov et al automate extraction given a known hierarchy shape, Auto-Echo infers the shape itself; where the side-channel literature recovers finer geometry, it does so with target-specific knowledge that this setting denies itself.

III. METHODOLOGY

Auto-Echo is a two-stage pipeline (Fig. 1). A native probe collects a latency-versus-working-set-size curve from user

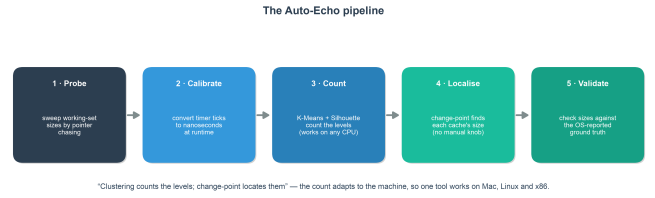


Fig. 1. The Auto-Echo pipeline. The probe emits a latency curve; clustering counts the levels; change-point detection localises each boundary; validation scores the result against operating-system ground truth. No stage receives the number of levels as input (Fig. 1).

space; an unsupervised inference stage then determines how many memory levels that curve contains and where each boundary lies. The stages are described in turn, but the design of the first is best understood from the failure that motivated it.

A. A Structural Failure, and What It Constrains

The first implementation followed the reference technique directly: flush a cache line, write it, then time the subsequent read. It failed on both architectures, and the reason was not the one initially assumed. Suspecting that ARM’s missing user-space flush was to blame, the same baseline was run on x86 *with* a working `clflush` and `rdtscp`. It failed there too, resolving two tiers on a four-level machine and reporting a mean “L1” latency of 178 ns — two orders of magnitude above that core’s true 1.6 ns L1.

The fault was structural rather than architectural. **Writing a line immediately before timing its read pulls that line into L1, so every measurement is an L1 hit by construction;** the flush accomplishes nothing because the store simply repopulates the line. A per-architecture port would not have helped. The measurement design itself had to satisfy three constraints simultaneously: never write before reading, never time a single access, and never require a flush primitive. Every subsequent design decision follows from one of the three.

B. The Working-Set-Size Pointer-Chase Probe

The probe measures load-to-use latency as a function of working-set size S . For each S , a page-aligned buffer is divided into slots one cache line apart, the line size being detected at run time (128 B on Apple Silicon, 64 B on x86). The slots are then linked into a **single random Hamiltonian cycle** by a Fisher–Yates shuffle (Knuth 1997) driven by a seeded `xorshift64` generator, so every sweep is reproducible from its seed. Constructing the cycle writes every slot, which pre-faults every page before timing begins.

Traversal is a data-dependent **pointer chase**: each load returns the address of the next. Three properties follow, and together they discharge all three constraints above.

The prefetcher is defeated without a flush. Addresses are unpredictable, so stride and stream engines cannot run ahead. A working set larger than a given cache overflows it *by construction*, which removes the need for any explicit

eviction instruction — essential on macOS/ARM64, where none is available to user space.

Memory-level parallelism is pinned to one. The address of load $n+1$ is the *result* of load n , so the out-of-order engine cannot issue them concurrently. This is what makes the measurement a latency rather than a bandwidth: independent loads would overlap across the miss-handling registers and report roughly the true latency divided by the achieved parallelism.

No hardware fence is required. Because the chase is already fully serialised by data dependency, there is nothing for a memory fence to order. The implementation uses only a *compiler* barrier around the timed region — empty inline assembly with a memory clobber, which emits no instruction and costs no cycles. A fence inside the loop would add real cycles to every hop, inside the window being measured. On x86 the timestamp is read with `rdtscp` rather than `rdtsc`, so prior instructions retire before the counter is sampled; on ARM64 an `isb` precedes the `cntvct_el0` read, which is an instruction-synchronisation barrier guarding the counter read, not a memory fence guarding the loads.

A single detail deserves emphasis because getting it wrong is silent. The value that keeps the chase alive is stored to a `void *volatile sink` — a *volatile pointer*, not a pointer to volatile. The volatile store is an observable side effect the compiler must emit, and emitting it requires the whole dependent chain. Declared the other way round the store is dead, and `-O3` removes the entire timed loop, reporting physically impossible latencies near zero rather than failing.

1) *Batch-Amortised Timing*: Apple Silicon’s counter advances at 24 MHz — roughly 41.7 ns per tick, from a `mach_timebase_info` rational of 125/3 — against an L1 hit of about 1.53 ns. The timer is 27 *times coarser than the signal*, so timing one access yields zero or one tick and nothing between.

The probe therefore never times a single access. It times $N \geq 2^{20}$ dependent hops in one window and divides by N . At 1.53 ns per hop the window spans ~ 1.6 ms, or $\sim 38,500$ ticks, so a ± 1 -tick quantisation is $\pm 0.0026\%$ of the total; and because N is an exact integer chosen by the program, dividing by it introduces no further uncertainty. The requirement this imposes is *stationarity*: per-hop cost must not drift within the window. Three mechanisms protect it. A warm-up traversal precedes timing, so cold misses and TLB fill do not fall inside the window. Each size is measured $R = 5$ times and the **minimum** retained, following `lmbench` practice (McVoy and Staelin 1996) on the argument that interference can only add time. And the sweep visits sizes in **seed-shuffled order** rather than ascending, which matters more than it appears: in ascending order the die warms monotonically with working-set size, so thermal drift would masquerade as a latency increase correlated with size — precisely the shape of a real cache boundary, and reproducible enough to survive a repeat run. Shuffling converts that systematic bias into noise.

2) *Runtime Timer Calibration*: Converting ticks to nanoseconds is itself a portability problem. The reference implementation parses the `cpu MHz` field of `/proc/cpuinfo`,

which is Linux-only and, more seriously, reports the *current turbo-scaled core clock* while `rdtsc` advances at the fixed invariant-TSC rate. Auto-Echo instead **calibrates at run time**, counting hardware ticks over a 50 ms interval of the OS monotonic clock (`clock_gettime`, or `QueryPerformanceCounter` on Windows).

The Intel machine supplies a quantitative reason this matters. Windows advertises the part at 2.40 GHz; calibration measures 0.383 ns/tick, a 2.61 GHz invariant TSC — a discrepancy of 8.8%. Both figures are correct, because they describe different clocks: 2.40 GHz is the performance-core base frequency, whereas the invariant TSC ticks at a reference rate deliberately decoupled from it. Had the framework taken the advertised value, every reported latency would be inflated by 8.8% while every detected *capacity* — a working-set size, which never passes through the tick conversion — remained correct. The error would have been confined to the one column no internal consistency check could expose.

C. Level Discovery: Count, then Localise

A cache of capacity C holds latency flat while $S \leq C$ and steps up once it overflows, so the number of memory levels is the number of *plateaus* in the latency-versus-log S curve, and each capacity is the size at the plateau-to-rise transition. Turning that observation into an algorithm runs immediately into a difficulty: both the clustering objective and the segmentation objective are monotonically non-increasing in the number of components, so neither can select its own. Minimising either returns the degenerate answer in which every observation is its own level.

Auto-Echo resolves this by splitting the problem between two stages that use different information, and giving each only the statistic sufficient for its own question.

Counting is order-ignoring. “How many latency regimes exist?” is a question about the *distribution* of latency values, not their arrangement, so the stage operates on the multiset of per-size log-latencies. The count is chosen by the **silhouette coefficient** (Rousseeuw 1987), which unlike inertia is not monotone in k : it scores cohesion against separation, and splitting a genuine plateau places two near-identical clusters adjacent to one another, collapsing the separation term. It therefore has an interior maximum and needs no penalty parameter.

The partition itself is solved *exactly*. On the real line an optimal k -means partition is necessarily contiguous in sorted order, which reduces the search from a Stirling number of set partitions to a choice of $k - 1$ cut points and admits an exact dynamic program in $O(kn^2)$ (Fisher 1958; Wang and Song 2011) (Appendix A). Prefix sums of x and x^2 make each candidate segment’s cost $O(1)$. Using the exact solver in place of Lloyd’s heuristic (Lloyd 1982) buys no accuracy on the measured curves — both reach the same k — but it makes optimality a property of the algorithm rather than an observation about two data sets, and it removes seeding, restarts and random state, so repeated runs are bit-identical. That matters because the evaluation reports variability across

sweeps, and solver-induced variance would contaminate a hardware measurement.

Localisation is order-respecting. “Where does each level end?” is a question about position along the size axis, so the second stage segments the ordered curve. Crucially it is given the count from the first stage and asked for exactly $k-1$ breakpoints, using a *count-constrained* dynamic program (Truong et al 2020). This is what keeps the pipeline free of thresholds: a penalty-based segmenter such as PELT (Killick et al 2012) would require choosing a penalty, and that penalty — not the data — would determine how many levels were reported. PELT is used in this work only as a sensitivity instrument, sweeping the penalty to confirm that the recovered count is not balanced on a knife-edge.

Segmentation operates on **log**-latency. Under a squared-error cost on raw nanoseconds the deepest transition dominates: on the M1 the L2→DRAM step is worth roughly 250 times more cost reduction than L1→L2, so a segmenter with a limited budget of breakpoints would spend them subdividing the DRAM region and never place the L1 boundary at all. Caches are laid out multiplicatively, so the physically meaningful quantity is a ratio; taking logarithms makes a factor-of-six step cost the same whether it occurs at 1.5 ns or at 130 ns.

Because the two stages consume different information, their agreement is informative rather than circular, and three further estimators — the elbow criterion, a Gaussian mixture and DBSCAN, all via scikit-learn (Pedregosa et al 2011) — are run alongside as independent cross-checks. None of them feeds the productive path.

D. Validation

Detected boundaries are matched to operating-system ground truth by optimal one-to-one (Hungarian) assignment, with a match declared when the capacities agree within a factor of two, i.e. one octave. Ground truth is read *per core* rather than per socket, which is essential on heterogeneous parts: the generic macOS keys report the efficiency cluster, and scoring a performance-core measurement against them would compare a Firestorm result to Icestorm hardware. Capacities are reported as the **median of per-sweep detections** rather than detected once on the aggregated curve, for a reason quantified in Section IV.

IV. RESULTS AND EVALUATION

A. Machines and Ground Truth

Two machines were measured: an **Apple M1** (ARM64, Firestorm performance core, 128 B line, 16 KiB base page) running macOS, and an **Intel Core i5-13450HX** (x86-64, Raptor Lake performance core, 64 B line, 4 KiB base page) running Windows. Both are consumer laptop parts, which bounds the generalisation claim and is returned to in Section V. Table I sets out what each of the three lenses of Section I reports for each machine.

Ground truth is read *per core*. On macOS this means `hw.perflvl0.*` rather than the generic `hw.*` keys, and the distinction is not cosmetic: the generic keys report

TABLE I
THE TWO MACHINES UNDER ALL THREE LENSES. THE VENDOR REPORT NAMES NO CACHE ON EITHER MACHINE; `LSTOPO` IS COMPLETE ON THE INTEL PART AND SILENT ABOUT THE M1’S SYSTEM-LEVEL CACHE; ONLY MEASUREMENT RESPONDS TO IT.

	Apple M1	Intel i5-13450HX
ISA, core	ARM64, Firestorm	x86-64, Raptor Lake
Line, base page	128 B, 16 KiB	64 B, 4 KiB
<i>Lens 1 — vendor system report</i>		
Cache data	none	none
<i>Lens 2 — lstopo / OS descriptors</i>		
L1d	128 KB	48 KB
L2	12 MB (cluster)	1280 KB per core
Last level	not reported	20 MB shared
<i>Lens 3 — Auto-Echo</i>		
Levels found	3	4 (2 MiB pages)
Deepest band	13.9 MiB	19.7 MiB

the *efficiency* cluster, so scoring the performance-core probe against them would compare a Firestorm measurement to Icestorm hardware and inflate the L1 error from +23.0% to +146% — outside the matching tolerance, turning a correct result into a reported failure. On Windows the equivalent is `GetLogicalProcessorInformationEx` filtered to the caches serving CPU 0, rather than the per-socket `Win32_CacheMemory` aggregate.

Because the evaluation scores the probe *against* this ground truth, the ground truth is itself a reference standard and needs verifying rather than assuming. It was therefore cross-checked against `hwloc` (Broquedis et al 2010), an independent implementation, on every machine. Two points follow. Where the operating system reports a cache, `hwloc` agrees with the framework’s own reader on every value, so the reference standard is verified. But `hwloc` reads the same interfaces, so where the OS is silent it is silent too: it is an independent *implementation*, not an independent *source*. The M1’s System-Level Cache is missing from `lstopo` exactly as it is missing from `sysctl`.

B. Discovered Hierarchies

Fig. 2 shows the recovered latency curves. The pipeline received no hint of how many levels either machine has.

On the **Apple M1** all five estimators agree on **three** levels. The L1 plateau sits at 1.53 ns and ends at a detected 157.5 KiB; a middle band at 9.19 ns extends to 13.9 MiB; DRAM follows at 130.43 ns. The middle band is the interesting one, because the operating system documents a 12 MiB L2 and nothing beyond it. The probe responds to capacity roughly 16% past that figure, and the only structure that can supply the difference is the ~8 MiB System-Level Cache that no interface reports. The L2 and the SLC are not separated: they resolve as a single band, and no estimator splits them. That is an honest limit of a one-dimensional latency sweep — two adjacent tiers with similar latency merge — and it is reported as one band rather than presented as two.

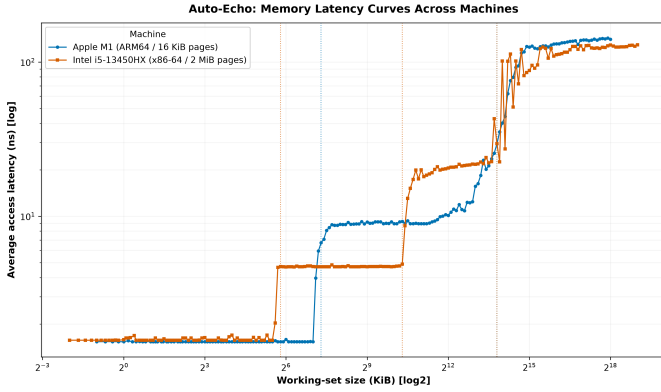


Fig. 2. Latency against working-set size for both machines on one log-log axis, with each machine’s automatically detected boundaries marked. The Intel curve is the 2 MiB huge-page run; under the default 4 KiB pages its L3 step is absent entirely (Table III).

TABLE II

CONSOLIDATED VALIDATION. GROUND TRUTH AS READ PER CORE; `hwloc` AS AN INDEPENDENT CROSS-CHECK; DETECTED CAPACITIES AS THE MEDIAN OF PER-SWEEP DETECTIONS. A MATCH REQUIRES AGREEMENT WITHIN A FACTOR OF TWO, I.E. AN ABSOLUTE ERROR OF AT MOST ONE OCTAVE. INTEL FIGURES ARE THE 2 MiB HUGE-PAGE RUN.

Machine	Cache	OS	Detected	Error	Match
Apple M1	L1d	128 KiB	157.5 KiB	+23.0%	✓
Apple M1	L2	12 MiB	13.9 MiB	+16.1%	✓
Apple M1	SLC	<i>n/r</i>	—	—	<i>n/s</i>
Intel i5	L1d	48 KiB	55.7 KiB	+16.0%	✓
Intel i5	L2	1.25 MiB	1.2 MiB	−1.5%	✓
Intel i5	L3	20 MiB	19.7 MiB	−1.5%	✓

n/r = not reported by any OS interface; *n/s* = not scorable, because no ground truth exists to score against. `hwloc` agrees with the OS on every row except SLC, where both are silent.

On the **Intel** part, under a 2 MiB huge-page allocation, the same unsupervised code path recovers **four** levels: L1 at 1.57 ns to 55.7 KiB, L2 at 4.71 ns to 1.2 MiB, L3 at 20.81 ns to 19.7 MiB, and DRAM at 122.36 ns. This is the framework’s first full L1/L2/L3/DRAM recovery, and it is reported with its provenance: it is a property of the huge-page run, not of default behaviour.

The two machines exercise the same code path on different ISAs, different line sizes (128 B against 64 B) and different page sizes (16 KiB against 4 KiB), with no per-machine configuration and no supplied level count. That is the substance of the architecture-agnostic claim.

C. Consolidated Validation

Table II places every documented cache on both machines against both reference sources: what the operating system says, what an independent implementation says, what Auto-Echo measured, and whether they agree.

Five documented caches, five matches, on two architectures. Recall is 2/2 on the M1 and 3/3 on the Intel part, at 100% precision on both: no detected knee is a false positive. Errors span −1.5% to +23.0%, and the two x86 inner levels land within 0.02 octaves of nominal.

TABLE III

THE SAME INTEL SILICON UNDER TWO PAGE SIZES. NOTHING ABOUT THE CACHE HIERARCHY DIFFERS BETWEEN COLUMNS; ONLY THE TRANSLATION PATH DOES.

	4 KiB pages	2 MiB huge pages
L3 boundary reported	~3.5 MiB	19.7 MiB
Deep-plateau latency	~143 ns	20.81 ns
Caches matched	2/3	3/3
Level count, per sweep	unstable (2–5)	4 (std 0.00)
Silhouette at selected <i>k</i>	0.885	0.933

Three points about this table deserve stating, because each answers a question a reader should be asking.

First, **the errors are one-signed on private caches, and that is a property of the estimator rather than noise.** Because the chase visits the working set in random order, a set moderately larger than the cache still enjoys high residency — at 157.5 KiB against a 128 KiB L1, roughly 81% of accesses still hit — so average latency rises gently past nominal rather than stepping. The detected plateau therefore extends slightly beyond true capacity, and it does so *upward* every time. The diagnosis is testable: an alternative *onset* estimator, which fires at the first departure from the plateau median, recovers the M1 L1 at exactly 128 KiB (+0.0%). It is not adopted as the default because it is unreliable elsewhere, under-reading three of the other four levels by 71–97% (Appendix C). This is an explicit accuracy-versus-robustness trade, decided for robustness.

Second, **the SLC row is deliberately unscorable rather than omitted.** No ground truth exists for it, so no error can be computed and reporting one would be fabrication. It is nevertheless the row that most justifies the method: the probe reaches 13.9 MiB against a documented 12 MiB L2, and only the unreported tier can account for the difference.

Third, **a match is a claim about an octave, not a percentage.** The defensible statement is that every documented cache has a detected boundary within a factor of two of its nominal capacity, and on that statement the framework is 5/5.

D. Page Size Decides Whether a Cache Is Visible

The Intel result above is conditional, and the condition is the sharpest finding in this work. Under the machine’s *default* 4 KiB pages, the documented 20 MiB L3 is not merely harder to detect — it is behaviourally unreachable.

The mechanism is TLB reach. A first-level data TLB of ~96 entries spans ~384 KiB under 4 KiB pages and ~192 MiB under 2 MiB pages — a factor of 512, exactly the page-size ratio. A 20 MiB working set therefore spans 5,120 pages against 96 entries, a 53-fold over-subscription. Long before the working set approaches L3 capacity, every dependent load misses in the TLB and triggers a page-table walk whose own accesses miss to DRAM. Past ~1.3 MiB the curve climbs steeply and *saturates* at a flat ~143 ns plateau by 4–5 MiB, staying there to 64 MiB. A saturated curve has no knee, so there is nothing for the inference stage to localise.

Four independent lines of evidence establish that the ~3.5 MiB band the segmenter reports under 4 KiB pages is

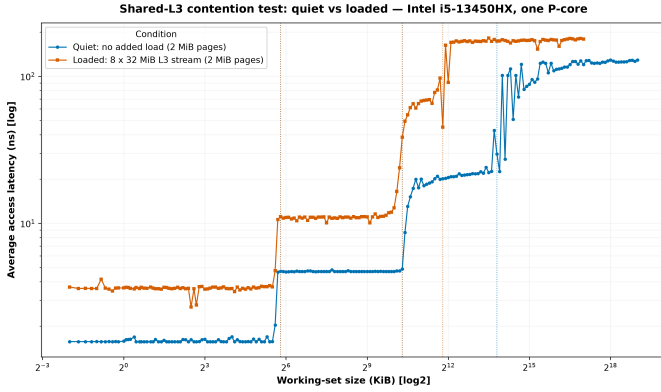


Fig. 3. The Intel L3 under load. Eight streaming workers on other physical cores drive the detected L3 boundary from 19.7 MiB to 3.5 MiB — a factor of 5.6 — with zero within-condition spread.

a TLB-transition artefact and not a cache. Its plateau latency is ~ 143 ns, seven times this part’s true L3 hit and essentially equal to DRAM — a structure that is not faster than what it fronts is not a cache. Its position is wrong by a factor of 5.7 against the documented 20 MiB, so the validation layer scores it as a miss. The curve never leaves that plateau out to 64 MiB, whereas a genuine 3.5 MiB cache would be followed by a further step into: under 2 MiB pages the artefact disappears and a feature matching the documented L3 to -1.5% appears in its place. The artefact was falsified by intervention, not inferred from its shape.

The wider consequence is the one drawn in Section I. Here the topology descriptor is entirely correct — there is a 20 MiB L3 — and the answer it gives is still not the answer a performance engineer needs, because reachability depends on the page size the allocation happens to use. No entry a topology tool could add would fix that, since the property being asked about belongs to the running system rather than to the die.

E. Contention Sets the Mappable Share of a Shared Cache

A second boundary condition concerns sharing. The Intel L3 is shared across all cores over a ring interconnect, so what a single probing core recovers is not the cache’s capacity but the share left to it. Fig. 3 shows the effect measured directly, with eight worker threads streaming 32 MiB buffers on other physical cores while the probe runs.

The detected L3 collapses from **19.7 MiB** (-1.5% of nominal) to a stable **3.5 MiB** (-83%) in every loaded sweep. The private L1 and L2 capacities are unchanged, exactly as expected: no other core can allocate into them, and the workers were pinned so that the probe’s own physical core stayed idle, excluding SMT interference within it.

Every latency also rises by a uniform factor of ~ 2.2 under load, and it would be easy to mistake that for the cause of the capacity change. It is not, and the two effects are separable by observation. The multiplier is already present at a **256 B** working set, where the shared L3 cannot be implicated at all;

a uniform, size-independent scaling is the signature of a *clock* effect rather than a cache effect. More importantly, the two act on different axes: a multiplier on latency rescales the curve vertically and cannot move the working-set size at which a plateau ends. The knee is a size, and sizes are frequency-invariant.

The clock effect is attributed to the loss of single-core turbo, but its magnitude is *not* claimed to be accounted for: this part’s 4.6 GHz single-core turbo against a 2.4 GHz base spans only $\sim 1.9\times$, so a $2.2\times$ rise implies operation below base frequency, an additional uncore component, or both. The framework cannot settle it, because the runtime calibration measures the *invariant* TSC, whose rate is by construction decoupled from the core clock — the same property that makes it a trustworthy timebase makes it useless as a tachometer. Resolving this needs `APERF/MPERF` or an OS performance counter, both privileged.

F. Robustness, and an External Check

Three further results bound the confidence in the above; each is summarised here and detailed in the appendices.

The level count does not depend on sampling resolution. Because the silhouette weights every observation equally, and the number of observations per level is fixed by the sweep grid rather than by the hardware, the selected count could in principle be an artefact of the experimenter’s chosen resolution. Re-run end to end at 5, 10 and 20 points per octave, the count is invariant on both machines — 3 on the M1 and 4 on the Intel part across a four-fold density change (Appendix C).

The inference layer transfers to a curve it did not produce. Applied unmodified to a latency curve measured by `lmbench’s lat_mem_rd` (McVoy and Staelin 1996) on the same silicon, the pipeline selects the correct level count and recovers both TLB-unmasked capacities (Appendix B). This tests the contribution — the inference stage — against prior art rather than only against itself.

Count stability is a property of a quiet machine, not of the part. On a contaminated run the productive counter over-counts, returning a modal seven levels with a standard deviation of 1.25 and ranking last of the five estimators, against a mean of 4.00 and standard deviation of 0.00 on the quiet ten-sweep run. This is a genuine limitation of the counter and is carried into Section V rather than left in the repository.

G. Data Availability

The latency curves underlying the M1 results, the Intel huge-page results, and the contention experiment are committed to the project repository as `wss_curve.csv` and `wss_curves_all.csv`, and every figure and table above derived from them is reproducible from that data. The 4 KiB-page Intel run of Table III is an exception: the machine was borrowed and is no longer available, and while the run’s outputs were recorded its raw curve was not retained. Those figures are therefore reported as measured but are not independently re-derivable from the repository, and they are

flagged as such rather than presented on the same footing as the rest.

V. DISCUSSION

The results support a narrower claim than “a tool that maps any cache hierarchy”, and the difference matters. What has been demonstrated is that an unsupervised pipeline, given no descriptor and no privileges, recovers every documented cache on two machines of different ISAs to within one octave, and responds to one undocumented cache that no interface reports. The boundaries on that claim are as much a result as the claim itself.

The evidence base is narrow. Two machines, both consumer laptop performance cores, is a thin foundation for a generalisation about architectures. The recovery is genuinely cross-ISA — 128 B and 64 B lines, 16 KiB and 4 KiB pages, ARM64 and x86-64, one code path — but a server part, a different-vendor core, or an efficiency core would each test the claim in a direction these two do not. The level *count* is also less unanimous than the headline suggests: the productive counter is stable and correct on both machines, but cross-estimator unanimity holds only on the M1, where all five agree on three levels. On the Intel part the elbow and cost-knee criteria return two against an expected four, and only the silhouette-based counter is right.

The two machines are not like-for-like. The M1 runs on a 16 KiB base page and the Intel on 4 KiB, so for any given working set the M1 places a quarter of the Intel’s demand on address translation. Because the large-page path is available only on Windows, the M1 has never been measured at any other page size — macOS on ARM64 rejects both `VM_FLAGS_SUPERPAGE_SIZE_2MB` and its permissive variant with `EINVAL`, superpages being an Intel-era facility the architecture does not honour. The cleaner deep region of the M1 curve therefore *cannot* be attributed to the architecture alone; part of it may simply be the larger page. This does not affect either machine’s validation against its own ground truth, which is what Section IV reports, but it forecloses the cross-machine comparison that would otherwise be tempting. The asymmetry is not an implementation gap that more effort would close: it is a difference in what the two platforms permit an unprivileged tool to do, which is the same class of obstacle that motivated the design.

Detected capacities carry a systematic bias, and its sign depends on sharing. For a private cache the bias is one-signed upward, for the residency reason given in Section IV. For a *shared* cache a second, opposing bias applies: a single probing core can only map the share left to it, so contention pulls the estimate down. On a quiet machine these partly cancel — part of why the Intel L3 reads a flattering -1.5% — and under load the downward term dominates completely, reaching -83% . A number whose sign depends on the ambient load of the rest of the system is not a constant of the hardware, which is why the L3 result is reported with its load condition attached.

No inferential statistics are claimed. Level counts are reported with a standard deviation and capacities with a non-

parametric min–max spread across sweeps; no hypothesis test is performed and no confidence interval is quoted. Ten sweeps is too few for a parametric interval, and reporting a standard error as though it were one would overstate what the data supports. Where a comparison is made — quiet against loaded — it rests on a five-fold effect with zero within-condition spread, not on a significance claim.

What is measured is load-to-use latency of a serialised chain, which includes base pipeline cost. Absolute values are therefore best read as relative differences between tiers rather than as architectural constants comparable across measurement methodologies.

Two threats are acknowledged rather than resolved. On Apple Silicon the probe relies on randomisation defeating the prefetcher, yet Vicarte et al (2022) demonstrated a data memory-dependent prefetcher on that exact part which follows pointer-like values — precisely the pattern this probe generates. The measured curves show clean, well-separated plateaus and a DRAM plateau at 130 ns consistent with unprefetched access, so the effect is not large enough here to flatten the staircase; but that is an empirical observation on one part, not a guarantee. Second, macOS exposes no user-space CPU-affinity API, so the probe requests a performance-cluster quality-of-service class rather than pinning. Calling that “pinning” would overstate it. Two checks show the bias took effect: `hwloc` confirms the clusters differ in cache geometry, and the recovered capacities score $+23.0\%$ and $+16.1\%$ against performance-core figures where the same measurements would read $+146\%$ and $+248\%$ against the efficiency cluster — far outside tolerance, so the result could not have come from the wrong cores.

VI. CONCLUSION AND FUTURE WORK

This paper asked whether a program can discover its machine’s cache hierarchy for itself, from timing alone, with no privileges, no vendor tables and no architecture-specific instructions. On two machines it can. A flush-free pointer-chase probe with batch-amortised, runtime-calibrated timing feeds an unsupervised stage in which exact one-dimensional clustering counts the memory levels and count-constrained change-point detection localises each capacity, with no threshold, no penalty and no supplied level count anywhere in the productive path. Every documented cache on both machines was recovered and matched within a factor of two: 2/2 on the Apple M1 and 3/3 on the Intel part, at 100% precision, against ground truth independently cross-checked with `hwloc`.

The three lenses of Section I are, in the end, the clearest statement of what this contributes. The commercial lens names no cache on either machine. The software lens is accurate wherever a vendor filled in the table — and on the Intel part it is entirely accurate, which this paper asserts rather than disputes; any argument for measurement resting on `lstopo` being *wrong* would be false. Its limit is that its chain of trust ends at what the operating system was told, so it inherits every omission, and it can only ever describe the hardware as designed. The physical lens answers a different question

— what this silicon does, on this run, at this page size, under this load — and the two machines show that the gap between those questions is not a technicality. On the M1 the gap is an entire ~ 8 MiB cache that no interface reports. On the Intel part it is a 20 MiB L3 that is documented, correct, and unreachable. Neither gap is visible to a table, and neither is discoverable by asking more politely. The case is for *complementing* the first two lenses, not replacing them.

Three extensions follow directly.

An AMD Zen part is the highest-value next measurement, because it differs from the Intel machine in precisely the variable the contention experiment isolates. Raptor Lake’s L3 is a single last-level cache shared over a ring interconnect, which is why eight streaming workers drove the recovered capacity down five-fold. Zen’s L3 is instead a victim cache private to each core complex, with inter-complex traffic crossing a separate fabric. The prediction is specific and falsifiable: workers pinned to the probe’s *own* complex should reproduce the under-read, while workers on a *different* complex should leave its slice largely intact, since they do not compete for the same physical cache. Confirming this would establish that the experiment measures cache *sharing* rather than system load; failing to confirm it would bound the finding to ring-based last-level caches. Either outcome is a result.

A page-size control on Apple Silicon would resolve the confound of Section V, but is blocked by the platform rather than by effort. It would require either a privileged allocation path or an unprivileged large-page interface that macOS on ARM64 does not currently provide.

Direct instrumentation of the two effects this work can only infer. Performance counters exposing DTLB-miss and page-walk-cycle events would confirm the translation mechanism of Table III independently of the page-size manipulation, and the APERF/MPERF pair would settle whether the uniform $2.2\times$ slowdown under load is entirely a core-frequency effect. Both are privileged, so both sit outside this project’s constraint — which is itself the point: the boundary of what unprivileged measurement can establish is exactly where those instruments become necessary.

REFERENCES

- Broquedis, F., Clet-Ortega, J., Moreaud, S., Furmento, N., Goglin, B., Mercier, G., Thibault, S. & Namyst, R. (2010). hwloc: a generic framework for managing hardware affinities in HPC applications. *Proceedings of the 18th Euromicro International Conference on Parallel, Distributed and Network-Based Processing*, 180–186.
- Bryant, R.E. & O’Hallaron, D.R. (2015). *Computer Systems: A Programmer’s Perspective*. 3rd ed. Boston: Pearson.
- Chen, B., Wang, Y., Shome, P., Fletcher, C.W., Kohlbrenner, D., Paccagnella, R. & Genkin, D. (2024). GoFetch: breaking constant-time cryptographic implementations using data memory-dependent prefetchers. *Proceedings of the 33rd USENIX Security Symposium*, 1117–1134.
- Drepper, U. (2007) What every programmer should know about memory. *LWN.net* [on-line]. Available from <https://lwn.net/Articles/250967/> [Accessed 4 August 2026].
- Fisher, W.D. (1958). On grouping for maximum homogeneity. *Journal of the American Statistical Association*, 53 (284), 789–798.
- Grønlund, A., Larsen, K.G., Mathiasen, A., Nielsen, J.S., Schneider, S. & Song, M. (2017) Fast exact k -means, k -medians and Bregman divergence clustering in 1D. *arXiv* [on-line]. Available from <https://arxiv.org/abs/1701.07204> [Accessed 4 August 2026].
- Intel Corporation (2023). *Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 2A: Instruction Set Reference*. Santa Clara, CA: Intel Corporation.
- Intel Corporation (2024) Intel Memory Latency Checker (Intel MLC) [on-line]. Available from <https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html> [Accessed 4 August 2026].
- Killick, R., Fearnhead, P. & Eckley, I.A. (2012). Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107 (500), 1590–1598.
- Knuth, D.E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3rd ed. Reading, MA: Addison-Wesley.
- Lloyd, S.P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28 (2), 129–137.
- McVoy, L. & Staelin, C. (1996). Imbench: portable tools for performance analysis. *Proceedings of the USENIX Annual Technical Conference*, 279–294.
- Osvik, D.A., Shamir, A. & Tromer, E. (2006). Cache attacks and countermeasures: the case of AES. *Topics in Cryptology — CT-RSA 2006*, LNCS 3860, 1–20. Berlin: Springer.
- Percival, C. (2005) Cache missing for fun and profit. *Proceedings of BSDCan 2005* [on-line]. Available from <https://www.daemonology.net/papers/cachemissing.pdf> [Accessed 4 August 2026].
- Rousseeuw, P.J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- Saavedra, R.H. & Smith, A.J. (1995). Measuring cache and TLB performance and their effect on benchmark run times. *IEEE Transactions on Computers*, 44 (10), 1223–1235.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. & Duchesnay, E. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Terpstra, D., Jagode, H., You, H. & Dongarra, J. (2010). Collecting performance data with PAPI-C. *Tools for High Performance Computing 2009*, 157–173. Berlin: Springer.
- Treibig, J., Hager, G. & Wellein, G. (2010). LIKWID: a lightweight performance-oriented tool suite for x86 multicore environments. *Proceedings of the 39th International*

- Conference on Parallel Processing Workshops*, 207–216.
- Truong, C., Oudre, L. & Vayatis, N. (2020). Selective review of offline change point detection methods. *Signal Processing*, 167, 107299.
- Vicarte, J.R.S., Flanders, M., Paccagnella, R., Garrett-Grossman, G., Morrison, A., Fletcher, C.W. & Kohlbrenner, D. (2022). Augury: using data memory-dependent prefetchers to leak data at rest. *Proceedings of the IEEE Symposium on Security and Privacy*, 1491–1505.
- Vila, P., Köpf, B. & Morales, J.F. (2019). Theory and practice of finding eviction sets. *Proceedings of the IEEE Symposium on Security and Privacy*, 39–54.
- Wang, H. & Song, M. (2011). Ckmeans.1d.dp: optimal k -means clustering in one dimension by dynamic programming. *The R Journal*, 3 (2), 29–33.
- Wulf, W.A. & McKee, S.A. (1995). Hitting the memory wall: implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23 (1), 20–24.
- Yarom, Y. & Falkner, K. (2014). FLUSH+RELOAD: a high resolution, low noise, L3 cache side-channel attack. *Proceedings of the 23rd USENIX Security Symposium*, 719–732.
- Yotov, K., Pingali, K. & Stodghill, P. (2005). Automatic measurement of memory hierarchy parameters. *ACM SIGMETRICS Performance Evaluation Review*, 33 (1), 181–192.

APPENDIX A

EXACT 1-D k -MEANS BY DYNAMIC PROGRAMMING

A. The contiguity lemma

For points on the real line, every optimal k -means partition is contiguous in sorted order. Suppose otherwise: there exist $x_a < x_b < x_c$ with $x_a, x_c \in C_1$ and $x_b \in C_2$. Since x_b lies within C_1 ’s span, and C_1 ’s centroid is a convex combination of its members and therefore also lies in $[x_a, x_c]$, moving x_b into C_1 strictly reduces the within-cluster sum of squares. The assumed partition was therefore not optimal.

The consequence is combinatorial. Without the lemma an optimal partition could be any partition of n points into k non-empty blocks, counted by the Stirling number of the second kind $S(n, k)$. With it, a partition is determined entirely by where the sorted sequence is cut, so the space collapses to $\binom{n-1}{k-1}$. For the Intel curve ($n = 202$, $k = 4$) that is a reduction from $S(202, 4) \approx 10^{119}$ to 1,313,400 — and, more usefully, the reduced problem has optimal substructure, so it need not be enumerated at all.

B. The recurrence

Let $D[c][j]$ be the minimal within-cluster sum of squares for partitioning the first j sorted points into c clusters, and $w(i, j)$ the cost of the single segment $x_i \dots x_{j-1}$. By the lemma the final cluster must be a suffix, so

$$D[c][j] = \min_{c-1 \leq i < j} (D[c-1][i] + w(i, j)), \quad D[0][0] = 0.$$

Segment cost is evaluated from the identity $\text{SSE}(i, j) = \sum x^2 - (\sum x)^2 / (j - i)$, so with prefix sums of x and x^2

each $w(i, j)$ is a single subtraction: $O(1)$ rather than $O(j - i)$, which reduces the total from $O(kn^3)$ to $O(kn^2)$. One fill of the table yields the optimal partition for *every* k up to the maximum, so the whole model-selection scan costs a single pass. Assignments are recovered by backtracking through the stored arg-minima.

Two implementation details matter. Segment costs are clamped at zero, because the algebraically non-negative identity above can return a small negative value through catastrophic cancellation on a near-zero-variance segment — an L1 plateau, where every point is nearly identical. Unclamped, that would make the program prefer spurious extra clusters inside a flat plateau, an artefact indistinguishable in the output from a real cache boundary. Second, Grønlund et al (2017) give an $O(n \log n)$ algorithm for the same problem using the objective’s monotone structure; it is not implemented here because at $n \approx 200$ the $O(kn^2)$ program completes in milliseconds against a probe that takes minutes, and the simpler formulation can be verified against brute-force enumeration.

C. Audit against Lloyd’s heuristic

The migration from Lloyd’s algorithm was audited rather than assumed. For every k in the search range and both measured curves, the globally optimal partition was compared against what Lloyd reached (Table IV).

TABLE IV
LLOYD’S HEURISTIC AGAINST THE EXACT DYNAMIC PROGRAM, PER k .

	Apple M1	Intel i5
Lloyd optimal at	$k = 2, 3, 4, 7, 8$	$k = 2, 3, 4, 5$
Lloyd sub-optimal at	$k = 5 (+2.9\%),$ $k = 6 (+1.4\%)$	$k = 6 (+2.0\%),$ $7 (+0.4\%), 8 (+0.5\%)$
Selected k : Lloyd	3 (sil. 0.894)	4 (sil. 0.935)
Selected k : exact	3 (sil. 0.894)	4 (sil. 0.935)

Lloyd did miss the global optimum, but only at $k \geq 5$ — beyond where the silhouette peaked on either machine — so the selected answer was unchanged. The exact solver therefore bought no accuracy on these curves, and the paper does not claim otherwise. What it bought is that optimality is a property of the algorithm rather than an observation about two data sets, and that with no seeding, restarts or random state, repeated runs are bit-identical.

APPENDIX B

EXTERNAL CROSS-CHECK AGAINST LMBENCH

The inference stage is the contribution, so it was tested against a curve it did not produce. lmbench’s `lat_mem_rd` (McVoy and Staelin 1996) was built and run on the same Intel silicon, and Auto-Echo’s unsupervised pipeline was applied to its output unmodified.

A. Do the two probes agree?

Both recover the same four-tier shape. Per-band medians (Table V) show lmbench reading uniformly higher, by $1.22\times$ at L1 rising to $1.54\times$ at L3.

TABLE V
PER-BAND MEDIAN LATENCY, AUTO-ECHO AGAINST LMBENCH, SAME MACHINE.

Band	Auto-Echo	lmbench	Ratio
L1 (to 55.7 KiB)	1.59 ns	1.93 ns	1.22×
L2 (59.7 KiB–1.2 MiB)	4.75 ns	5.99 ns	1.26×
L3 (1.3–13.9 MiB)	22.94 ns	35.35 ns	1.54×
DRAM (14.9–512 MiB)	123.25 ns	160.16 ns	1.30×

The offset is not a uniform scale factor, and an earlier draft’s claim that the two curves run parallel is withdrawn. Point-wise ratios within the L2 band climb steadily rather than staying flat, and the reason is the confounder of Section IV: the Auto-Echo curve is a 2 MiB huge-page run whose enlarged TLB reach removes page-walk cost, whereas lmbench ran on 4 KiB pages under WSL2. On 4 KiB pages an 832 KiB working set already spans 208 pages against a ~ 96 -entry first-level DTLB, so translation cost enters lmbench’s curve *inside* the L2 band — precisely where the divergence begins. The L1 band, at 24 pages or fewer, is the only region genuinely below TLB pressure for both, and it is the only region where the ratio is stable.

B. Does the inference transfer?

The stronger test is whether the pipeline, applied to lmbench’s curve, recovers the hierarchy. It selects the correct level count and localises the two TLB-unmasked capacities to +16.7% and −10.0%, both matches. The L3 is recovered at 6.5 MiB against a nominal 20 MiB — a miss at −68.2% — which is the expected consequence of applying the inference to a 4 KiB-page curve whose L3 knee has been pulled inward by exactly the mechanism Table III isolates. The inference layer transfers; the input’s page size still governs what is there to be found.

APPENDIX C

ESTIMATOR AND SAMPLING-DENSITY ROBUSTNESS

A. Sampling density

Because the silhouette weights every observation equally, and the number of observations falling in each level is fixed by the sweep’s geometric grid rather than by the hardware, the selected count could in principle be an artefact of the experimenter’s chosen resolution. This would make the central claim circular, so it was tested directly: the probe was re-run end to end at 5, 10 and 20 points per octave on both machines — each row an independent sweep, not a re-analysis.

The count is invariant across a four-fold change in density on both machines. The 20-points-per-octave Intel row lies *above* the original source grid and so could not have been produced by subsampling an existing curve, which closes the obvious objection to the test.

B. Capacity estimators

Four capacity estimators are selectable. *Edge* takes the last working-set size on the plateau; *midpoint* the geometric mean

TABLE VI
SELECTED LEVEL COUNT AGAINST SWEEP RESOLUTION.

Machine	Pts/oct	Points	Selected k	Sil.
Apple M1	5	94	3	0.912
Apple M1	10	182	3	0.898
Apple M1	20	348	3	0.896
Intel i5	5	104	4	0.929
Intel i5	10	202	4	0.934
Intel i5	20	388	4	0.932

of the last-fit and first-miss sizes; *onset* the first departure from the plateau median; and *hybrid* applies onset where the plateau is flat and edge otherwise. Table VII gives absolute error against per-core ground truth.

TABLE VII
CAPACITY-ESTIMATOR ERROR AGAINST PER-CORE GROUND TRUTH. EDGE IS THE DEFAULT. NOTE THE L3 ROW CARRIES THE AGGREGATE-CURVE ARTEFACT DISCUSSED BELOW, NOT THE PER-SWEEP MEDIAN REPORTED IN THE BODY.

Level	Edge	Mid	Onset	Hybrid
M1 — L1 (128 KiB)	+23.0	+27.4	+0.0	+0.0
M1 — L2+SLC (12 MiB)	+16.1	+20.2	−71.0	+16.1
Intel — L1 (48 KiB)	+16.0	+20.1	−96.9	−96.9
Intel — L2 (1.25 MiB)	−1.5	+2.0	−8.1	−8.1
Intel — L3 (20 MiB)	−30.4	−27.9	−77.0	−30.4

The result is a clear accuracy-versus-robustness trade. Onset is *exact* on the M1 L1, which confirms that the +23.0% edge error is a genuine soft-knee artefact and that the physical knee sits at the documented capacity. But it is unreliable everywhere else, from a tolerable −8.1% to catastrophic under-reads on three of the remaining four levels, because on a sloping plateau — the M1’s merged L2+SLC band, or a contended L3 — it fires almost immediately. Edge is retained as the default on those grounds, and the trade is reported rather than hidden.

C. Aggregation and the choice of statistic

One further result belongs here because it corrects an earlier version of this work. Detecting levels once on the aggregated `wss_curve.csv` — which is the *minimum* over sweeps at each size — placed the Intel L3 at 13.9 MiB (−30.4%), the figure appearing in Table VII. Detecting in each sweep independently and taking the median places it at 19.7 MiB (−1.5%), and *nine of the ten individual sweeps* give the latter. The minimum is the right statistic for a *latency*, because interference can only add time; it is the wrong statistic for a *boundary*, because a lower envelope in a noisy transition drags the apparent end of a plateau inward. The body reports per-sweep medians throughout for this reason.

APPENDIX D GROUND-TRUTH PROVENANCE

Table VIII records what each source reports for the probed core on each machine, including the *generic* interfaces the framework deliberately does not use.

TABLE VIII
GROUND-TRUTH PROVENANCE. THE THIRD COLUMN IS THE GENERIC
INTERFACE THAT WOULD HAVE BEEN READ HAD THE PER-CORE
DISTINCTION BEEN OVERLOOKED.

Machine	Cache	Used	Generic (not used)
Apple M1	L1d	128 KiB	64 KiB (E-cluster)
Apple M1	L2	12 MiB	4 MiB (E-cluster)
Apple M1	SLC	<i>not reported</i>	—
Intel i5	L1d	48 KiB	288 KiB (per socket)
Intel i5	L2	1.25 MiB	7680 KiB (per socket)
Intel i5	L3	20 MiB	— (shared; coincides)

The error the generic keys would have introduced is not subtle. On the M1 the detected L1 boundary of 157.5 KiB scores +23.0% against the Firestorm figure of 128 KiB, but would score +146% against the Icestorm figure of 64 KiB — outside the factor-of-two tolerance, so the evaluation would report a validation failure rather than a match. The same reasoning motivates using `GetLogicalProcessorInformationEx` rather than the per-socket `Win32_CacheMemory` aggregate on Windows.

Two further observations come from the `hwloc` cross-check. The two heterogeneous parts number their cores in *opposite* orders: the Intel’s six performance cores come first, whereas the M1’s logical CPUs 0–3 are efficiency cores. A single portable pinning rule would therefore have selected the wrong core type on one of the two machines, which is why the Windows path uses an affinity mask and the macOS path a performance-cluster quality-of-service hint. And `hwloc` reports the M1’s installed memory exactly (8192 MB against 8 GB) while reporting 6928 MB against 24.0 GB on the Intel machine, because its Windows backend returns memory *available* on the node rather than installed capacity — a difference in the platform backend rather than in the tool, which the macOS row serves to establish.